

A Tabu Search algorithm for the Probabilistic Orienteering Problem

Xiaochen Chou^a, Luca Maria Gambardella^b, Roberto Montemanni^{c,*}

^a Dalle Molle Institute for Artificial Intelligence, Galleria 2, CH-6928 Manno, Switzerland

^b Faculty of Informatics, Università della Svizzera Italiana, Via Buffi 13, 6900 Lugano, Switzerland

^c Department of Sciences and Methods for Engineering, University of Modena and Reggio Emilia, Via Amendola 2, 42122 Reggio Emilia, Italy

ARTICLE INFO

Article history:

Received 2 April 2020

Revised 2 September 2020

Accepted 23 September 2020

Available online 2 October 2020

Keywords:

Probabilistic Orienteering Problem

Metaheuristic algorithms

Tabu Search

Monte Carlo Sampling

ABSTRACT

The Orienteering Problem is a routing problem aiming at selecting a subset of a given set of customers to be visited within a given time budget, so that a total revenue is maximized. Multiple variants of the problem have been studied. The Probabilistic Orienteering Problem is one of these variants, where customers will require a visit according to a certain given probability. Stochasticity makes the model more practical, but concurrently more difficult to solve. Tabu Search is a method widely used in combinatorial optimization problems to escape from local optima in heuristic local search procedures. In this work, we solve the Probabilistic Orienteering Problem by embedding a Monte Carlo evaluator into a Tabu Search algorithm, exploiting their interaction in an innovative way. A detailed computational study of the new approach is presented, with the aim of studying the performance of the metaheuristic algorithm in terms of precision and speed, while positioning the new method within the existing literature.

© 2020 Elsevier Ltd. All rights reserved.

1. Introduction

Stochastic Combinatorial Problems have drawn the attention of many researchers in recent years. With respect to classic optimization approaches, uncertainty factors are taken into account, thus the resulting models are more realistic. However, stochastic problems are also more difficult to solve. The challenge is both on modelling and computation aspects: exact methods usually work only for small instances, besides, there are several problems with no closed-form expression or hard-to-compute objective functions. Designing efficient (meta) heuristics to produce high quality solutions in a reasonable amount of time is an actual research goal, and we develop such a method for the Probabilistic Orienteering Problem.

The Probabilistic Orienteering Problem (POP) has been first introduced in Angelelli et al. (2017). It is a variant of the Orienteering Problem (OP) (Vansteenwegen et al., 2011) where customers are requiring visits with certain probabilities. In the OP, there is a set of customers at different locations and with associated prizes. A prize is collected only if the respective customer is visited. The aim is to maximize the total prize collected within a given time budget. In the POP, the potential customers will require a visit according to a given probability, correspondingly, a prize can be

collected only when the associated customer actually requiring a visit and is served before the given deadline. Thus, the target of the POP is to construct a planning that consists in selecting a subset of the given customers within a given time budget, in such a way that the respective expected prize collected is maximized and the expected total travel time cost is minimized. Given a tour, the analytical approximation of the expected total travel time cost requires cumulative products considering all combinations in which each customer requires a visit or not. In Chou et al. (2018) a Monte Carlo Sampling approximation method for the POP has also been proposed, and it has been proven to be fast and effective method when used inside a heuristic solver for the POP. In this work, we propose a Tabu Search algorithm embedding a Monte Carlo evaluator to solve the POP. An innovative integration of the two components allows for simple and effective algorithms. With this technique, we provide solutions for the POP with low error in a very short time.

The paper is organized as follows. A formal definition of the POP is provided in Section 2. A review of the literature focusing on the POP and on common methods used for several related stochastic combinatorial problems is presented in Section 3. In Section 4, we describe an Tabu Search algorithm based on a Monte Carlo Approximation method for estimating the objective function value. Experimental results on the performance of the metaheuristic algorithm proposed are presented in Section 5, with an analysis of limitation and advantages of our approach with respect to other methods from the literature. Conclusions are finally drawn in Section 6.

* Corresponding author.

E-mail addresses: xiaochen@idsia.ch (X. Chou), luca.gambardella@usi.ch (L.M. Gambardella), roberto.montemanni@unimore.it (R. Montemanni).

2. The Probabilistic Orienteering Problem

The Probabilistic Orienteering Problem answers the following question: Given a time budget and a list of customers at different locations with different probabilities of requiring a visit and prizes to be collected, what is the best subset of customers to visit (and in which order) so that the expected total collected prize is maximized and the travel time is concurrently minimized? This problem, faced by companies while organising their day-ahead plan or general users having to take real time decisions (Chou et al., 2019b), is a variation of the Orienteering Problem which takes uncertainties of the customers' requirements into consideration, with uncertainty modelled through probabilities.

As described in Vansteenwegen et al. (2011), the Orienteering Problem can be defined on a graph $G = (V, A)$ where $V = \{v_1, \dots, v_N\}$ is the node set representing customers and A is the arc set representing the paths between pairs of customers. A prize p_i is associated with each node $v_i \in V$, a travel time t_{ij} is associated with each arc $a_{ij} \in A$ and a deadline D is finally provided. A feasible tour σ is defined as a sequence of customers that can be visited before the deadline incurs. With this notation, the OP can be described as the problem of finding the tour that collect the largest possible amount of prizes from customer within the given deadline among all feasible tours (Vansteenwegen et al., 2011). This of course implies a selection of the shortest tour among the selected customers. The objective function of the OP can be reduced therefore to the maximization of the following quantity $o(\sigma)$, given a feasible (partial) tour σ not exceeding the deadline D :

$$o(\sigma) = \sum_{i \in \sigma} p_i \quad (1)$$

Differently from the OP, the objective function of the POP is no longer a simple sum. Due to its probabilistic nature, customers that offers a large prize may require a visit with a very low probability. With this consideration, the POP is defined as follows.

A POP instance contains the information (V, t, π, D, p) where:

- $V = \{0, 1 \dots n, n+1\}$ is the set of n customers (nodes) with the depot being node 0 and the destination being node $n+1$.
- t_{ij} is the deterministic travelling time from customer i to customer j defined, for all $i, j \in V$.
- π_i is the probability of a customer $i \in V$ to require a visit and is modeled by a Bernoulli variable $b_i = \{0, 1\}$. b_i takes value 1 with an independent probability π_i for each customer. Depot and destination are required to be visited by definition, therefore $\pi_0 = \pi_{n+1} = 1$.
- D is the global deadline.
- p_i is the prize collected when a customer $i \in V$ is served before the global deadline D . There is no prize for depot and destination, therefore $p_0 = p_{n+1} = 0$.

Formally, we define a feasible solution $\sigma = (i_0 = 0, i_1, i_2, \dots, i_q, i_{q+1} = 0)$ as a tour starting from the depot, allowing to serve a sequence of q customers even when all customers on the route require visits and return to the depot before the global deadline D . A complete tour $\tau = (0 = i_0, i_1, i_2, \dots, i_n, i_{n+1} = n+1)$ is defined as a sequence of n customers, plus the depot and the destination node. For a given complete tour τ , the associated prize is $P(\tau)$ and the associated travel time is $T(\tau)$. The POP is a combinatorial optimization problem with the objective of maximizing the total expected prizes collected while visiting customers before the given deadline, minus a measure of the total expected travel time. The last factor helps to make a choice when several tours are associated with a same prize, and in this case we pick the one associated with

smaller travel cost. Note that the travel time component is not present in the classic formulation of the OP described above.

The objective function of the POP is to retrieve a feasible solution σ among all possible tours that maximize the following quantity:

$$u(\sigma) = \mathbb{E}[P(\sigma)] - C\mathbb{E}[T(\sigma)] \quad (2)$$

where the difference between the expected total prize and the expected total travel time is multiplied by a given coefficient C that is normally set by the decision maker (Angelelli et al., 2017).

3. Literature review

As shown in Bérubé et al. (2009) and Feillet et al. (2005), the OP can be formulated as a Travelling Salesman Problem with Profits. The problem has been proved to be NP-hard (Golden et al., 1987). To solve the OP, a brand-and-bound method is used in Laporte and Martello (1990) and Ramesh et al. (1992). Improved approaches based on branch-and-cut have been proposed in Fischetti et al. (1998) and Gengreau et al. (1998). There has been continuous improvements to the solving methods over the years, and a classification of the exact algorithms and details about the applied techniques can be found in Feillet et al. (2005). The OP arises in several applications, a recent application is the Mobile Tourist Guide (Souffriau et al., 2008), which makes feasible plan for tourists to visit the most valuable attractions among all the places that they want to visit in the limited time of the trip. Such planning problems are called Tourist Trip Design Problems (TTDP) (Vansteenwegen and Oudheusden, 2007). Similar Tourist Trip problems of selecting the most interesting combination of attractions is mentioned in Chou et al. (2019b) and Wang et al. (2008). In general, the application of the TTDP requires high quality solutions in a short calculation time, which help to make real-time decisions.

There are several deterministic extensions to the OP. The Team Orienteering Problem (TOP) (Chao et al., 1996; Tang and Miller-Hooks, 2005a; Archetti et al., 2007) considers multiple path limited by the deadline and the goal is to maximise the total prize collected. The Orienteering Problem with Time Windows (OPTW) and the Team Orienteering Problem with Time Windows (TOPTW) add time window constraints for the customers, and a visit to a customer can only starts during this time window. The OPTW and TOPTW have been solved with Ant Colony optimization in Montemanni and Gambardella (2009) and Montemanni et al. (2011), and Iterated Local Search metaheuristic in Vansteenwegen et al. (2009). Another interesting variation of the OP is the Clustered Orienteering Problem (COP) proposed by Angelelli et al. (2014), where each profit is associated with a cluster of customers and is gained only when all customers in the cluster are served.

The probabilistic variations of the OP are mainly considering uncertainty on time and/or profit. From the objective perspective, the POP we study in this work is similar to the Probabilistic Travelling Salesman Problem with deadline (PTSPD) (Campbell and Thomas, 2008). In the PTSPD, a minimum expected cost a priori tour is to be found through a set of customers with probabilities of requiring service. Each customer has a deadline, and service at each customer should begin at or before its deadline. The objective is to minimize the expected travel time and the expected penalty caused by violation of the deadline. In Campbell and Thomas (2008), different models are considered such as paying a penalty for any violation of a deadline, or paying penalties for skipping the customers whose deadline would be violated. In both the POP and the PTSPD, the stochasticity is on the probability of a customer requiring service. The main dif-

ference is that a POP feasible solution is defined when every customer on the tour must be visited before a global deadline, while in PTSPD customers can be visited after its deadline with a penalty. Other variants of the OP with stochasticity on service or travel time have been studied in Tang and Miller-Hooks (2005b) and Campbell et al. (2011). An OP with uncertain collected prizes has been considered in Ilhan et al. (2008). From the application perspective, a tourism-related practical application of the POP has been studied in Chou et al. (2019b). Since the simplest TTDP model has been proved to be identical with the OP, by taking the availability of the points of interests into account, a Probabilistic Tourist Trip Design Problem with personalized customization is designed and can be solved with algorithms for the POP. The models can also be extended with extra feature. For example a Probabilistic Team Orienteering Problem (PTOP) based on the TOP can be open to more applications, such as planning a different tour for each day during the length of the stay, or determine same or separate tours for a tour group, regarding the different satisfaction scores given by each member in the group and the availability of the points of interests on the list.

The POP inherits the NP-hardness from the OP. To solve the POP, a MILP model has been proposed and the problem is solved by branch-and-cut methods in Angelelli et al. (2017). Since the solving with a basic branch-and-cut method is computationally demanding, three matheuristic methods to reduce the solution time (detailed in Section 5) have also been also studied in Angelelli et al. (2017). The use of Monte Carlo Sampling techniques in the context of the POP has been studied in Chou et al. (2018). A fast objective function cost approximator for the problem using Monte Carlo Sampling method has been proposed. Similar uses of the Monte Carlo Sampling approach for stochastic/probabilistic vehicle routing problems can be found in Weyland et al. (2013) and Papapanagiotou et al. (2015). Given the characteristics of an instance, a Machine Learning-based study on estimating the best number of samples to use in such a Monte Carlo Sampling evaluator for the POP can be found in Montemanni et al. (2018).

Local search methods provide fast and efficient heuristic solutions for problems with high computational complexity consistently (Hoos and Stützle, 2005). However, a standard technique only searching in the neighborhood of the solution may get trapped in local extrema (Shaw, 1997). To take care of this undesired phenomenon, random restart strategies applied on top of the adopted local search are normally used, in order to increase the probability of success in searching procedures. In Chou et al. (2019a) a solver for the POP using Random Restart 2-opt local search, which proved to be fast and efficient for small instances has been proposed. A study on improving the performance of the solver by modifying the generation of re-initialising solutions has been presented in Chou et al. (2020).

Tabu Search is a metaheuristic search technique that has been widely used in combinatorial optimization problems since it was introduced by Glover (1986). It is another effective method to deal with local extrema. The basic idea is to accept a sequence of non-improving solutions to escape from local extrema. This is achieved by the use of an explicit memory called Tabu list during the search process. It avoids visiting solutions that have been visited recently. A variety of implementations of Tabu Search in the context of the Travelling Salesman Problem (TSP) and related problems is reviewed and compared in Basu (2012). Other methods such as Simulated Annealing (Kirkpatrick et al., 1983), Genetic Algorithms (Holland, 1992) and Ant Colony Optimization (Dorigo and Gambardella, 1997; Gambardella et al., 2012) are other techniques frequently used to escape from local extrema.

4. A Tabu Search heuristic algorithm

In this Section, we introduce a Tabu Search (TS) Algorithm for the POP. The method is designed to be fast, simple, and with reasonably small optimality gaps.

Normally, for problems involving the choice of a subset of the customers, it is common to have several heuristic operators, some of which handling the selection of the customers and some others their sequencing (see, for example Campbell and Thomas, 2008). In our case, the method is very simple. We only have a 2-opt heuristic operator, being the algorithm itself always treating complete tours covering all the customers. The choice of the subset of customers to be included in the feasible POP solution is delegated to the Monte Carlo Sampling component, which is also in charge of approximating the objective function value of a given tour. Such a design choice not only limits the number of the required heuristic operators, but also exploits the synergy between the metaheuristic and the embedded Monte Carlo approximator.

4.1. The role of Monte Carlo Sampling

From a computational perspective, solving the POP is computationally demanding. As a variation of the OP, the POP is also NP-hard (Keller, 1989). Given a feasible tour, the objective function value can be evaluated in quadratic time. However, optimizing the feasible tour is a complex task. The MILP model presented in Angelelli et al. (2017) based on feasible solutions has an exponential number of constraints.

We use the Monte Carlo Sampling method as an interrelated part of a simple heuristic to solve the POP in a simple and effective manner.

The sampling method works as follows: For a given complete tour τ with n customers, according to the probability of each customer to require a visit, a set of s scenarios is generated. Each scenario is a fully connected graph with a fixed set of customers requiring a visit. For each scenario $i \in \{1 \dots cs\}$, a deterministic objective function value can be evaluated in linear time as $u_d(\tau_i) = P(\tau_i) - C \cdot T(\tau_i)$, thus the objective function value $u(\tau)$ of a complete tour τ can be approximated by averaging these values in time $O(ns)$.

Fig. 1 shows an example of generating s scenarios for a given complete tour τ . Each block represents a scenario and each dot in

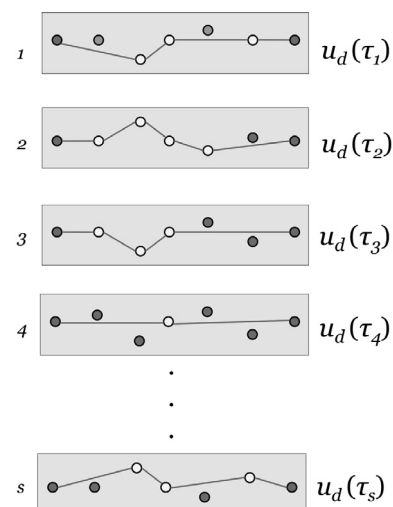


Fig. 1. example of s scenarios generated from a tour τ .

the block represents a node, which can be either a customer or the depot/destination. In each scenario, whether a customer requires a visit or not is deterministic. When a customer i does not require a visit, he will be skipped while travelling on the tour τ , and the truck moves from the previous customer directly to the next customer that requires a visit. For example in scenario 1, customer 1 and 4 do not require a visit, therefore, a tour $\tau_1 = [0, 2, 3, 5, 6]$ is obtained, and straightforwardly, its objective function value $u_d(\tau_1)$ can be calculated. Similarly, a total of s scenarios are generated regarding the probability of each customer in tour τ requires a visit. With $u_d(\tau_i)$ being the deterministic cost of the tour τ_i adapted to the deterministic scenario i , the approximation value for $u(\tau)$ is given by the average objective function value of the deterministic costs of all the scenarios considered:

$$u(\tau) \approx \frac{\sum_{i=1}^s u_d(\tau_i)}{s} \quad (3)$$

Due to the heuristic nature of the approach, the approximation of the objective function value $u(\tau)$ is more accurate when more scenarios are used, but this requires more computational time. A detailed study on experiments for understanding promising values of the parameter s can be found in Chou et al. (2018) and Montemanni et al. (2018). In general, the approximation error for $u(\tau)$ is less than 1% when $s \geq 50$, and the computational speed is over 10^5 evaluations per second when $s \leq 250$ on a normal modern personal computer. A number of samples in this interval is a considerable choice. When the Monte Carlo Approximation is embedded in a metaheuristic algorithm for the POP, high speed is desirable for a better exploration of the searching space, while precision is less crucial. Therefore, $s = 50$ is adopted as setting for all the experiments presented in this work.

By design, the TS algorithm treats complete tours touching all customers, while a POP solution only visits a subset of customers that can be served before the deadline D . The Monte Carlo Sampling component is also delegated to extract a POP solution $\sigma(\tau)$ as a prefix of a complete tour τ . By the definition of the problem, the travel time of going from the last visited customer back to the depot has always to be considered. Due to this feature, when a complete tour $\tau = (i_0 = 0, i_1, i_2, \dots, i_n, i_{n+1} = n + 1)$ is being evaluated over a given deterministic scenario s , the tour stops at the last customer i_q ($q \leq n$) before the deadline D is incurred, and then goes directly to the destination node. The feasible POP solution selected out of a complete tour can also be easily obtained in this way by supposing all customers on the route require visits. However in certain situation, it is possible that some customers visited within the deadline are not offering enough prize to make an increase on profit. Therefore, to make a smarter selection that is consistent with the objective function, we take the best feasible POP solution $\sigma(\tau)$ selected out of a complete tour τ as the one with the peak value of (3) encountered during the evaluation of τ , denoted as $u(\sigma(\tau))$. This coincide with the fact that during the evaluation of a complete tour τ , after the deadline is incurred only the travel time component increases, but no more prize is collected.

In conclusion, the Monte Carlo Approximation component of the TS algorithm receives in input a complete tour τ and outputs:

- a POP feasible solution $\sigma(\tau)$ extracted from the complete tour τ ;
- an approximation for the objective function value $u(\tau)$ of the complete tour τ ;
- an approximation for the objective function value $u(\sigma(\tau))$ of the POP solution $\sigma(\tau)$.

4.2. Generation of an initial complete tour

As described in Basu (2012), the choice of an initial tour plays an important role in retrieving good solutions using local search based metaheuristics like Tabu Search. *Random Insertion* and *Nearest Neighbour* are common and simple ways to generate an initial tour (Bräysy and Gendreau, 2015). They both start from a partial tour initially containing only the depot 0. By using a *Random Insertion* method, we insert at random customers not included in the partial tour into the tour without forming sub-tours in between until all customers are covered, and we add the destination $n + 1$. In the *Nearest Neighbour* method, we search for the nearest unvisited customer to the last customer of the partial tour, and we add it to the partial tour. The procedure is repeated on the incrementally larger partial solution until all the customers are covered, the destination $n + 1$ is finally added.

To keep the best features of each initialising method, a k -Nearest Random Insertion method inspired by Mbiadou Saleu et al. (2018) is presented in Chou et al. (2020). Instead of picking the nearest neighbour at each step, a random insertion among the k closest ones is proceed when generating the initial tour. The larger the k value is, the more randomized the initial tour will be, meanwhile longer time are possibly required for convergence. As proved in Chou et al. (2020), rapid convergence and enough randomness can be retained at the same time with appropriate tuning for parameter k . Experimental results in Chou et al. (2020) show that $k = 2$ works well for the POP, therefore we adopt this value for the experiments in this work.

We have also considered other simple initialising solution strategies such as using the optimal TSP solution. However, the solutions generated were similar to those of the *Nearest Neighbour* method, with the issue mentioned earlier even more amplified. Thus we did not use it in this work.

4.3. A 2-opt local search heuristic

In Tabu Search we move from one complete tour to another in the searching process, based on the concept of neighbourhood of a given tour. The neighbourhood is defined by a local search method.

The local search method we use in this work is a 2-opt heuristic (Blazinkas and Misevicius, 2011), which deletes two non-adjacent edges of a tour and reconnects the two paths resulting from this break in the other possible way without creating sub-tours. To be more specific, for a given tour $\tau = (i_0 = 0, i_1, i_2, \dots, i_n, i_{n+1})$, we pick all possible first customer j from $\{i_1, \dots, i_{n-1}\}$ and all possible second customer k from $\{i_{j+1}, \dots, i_n\}$, then reverse the path between these two customers, obtaining $\tau' = (i_0, i_1, \dots, i_j, i_k, i_{k-1}, \dots, i_{j+1}, i_{k+1}, \dots, i_n, i_{n+1})$. The sequence of new tours generated from τ is the neighborhood of τ , denoted as $\mathcal{N}(\tau)$. The best non-tabu (see Section 4.4) new tour among those in $\mathcal{N}(\tau)$ will become the new incumbent solution.

As described in Section 4.1, feasible POP solutions are extracted from complete tours containing only customers that can be visited before the deadline. Accordingly, several complete tours obtained by swapping between pairs of customers that are both visited after (especially far after) the deadline actually result in the same POP solution with high probability. Therefore in our algorithm, the local search proceeds in two steps to increase effectiveness (details in Section 4.4). Regarding this implementation, both the neighbourhood of a complete tour τ and that of its corresponding feasible solution $\sigma(\tau)$ need to be considered.

Some theoretical results about the number of complete tours and feasible solutions present in the 2-opt neighbours are as fol-

lows. They will be used in Section 4.4 for some considerations about the length of the Tabu list.

Given a complete tour τ with n customers, the total number of complete tours contained in the 2-opt neighbourhood $\mathcal{N}(\tau)$ is determined by the number of combinations of picking 2 customers out of n :

$$|\mathcal{N}(\tau)| = \frac{n(n-1)}{2} \quad (4)$$

Given a feasible solution $\sigma(\tau)$, the total number of feasible solutions contained in the 2-opt neighbourhood $\mathcal{N}(\sigma(\tau))$ is determined by the number of combinations of picking 2 customers out of q , with $q = |\sigma(\tau)|$ being the number of customers in the feasible solution $\sigma(\tau)$ extracted from the complete tour τ :

$$|\mathcal{N}(\sigma(\tau))| = \frac{q(q-1)}{2} \quad (5)$$

4.4. Memory structure

A memory structure is used in Tabu Search to prevent the searching process from re-visiting tours that have been visited recently. This should prevent the method to get trapped in local extrema. This structure filters out some tours and defines which tours in the neighborhood $\mathcal{N}(\tau)$ can be explored by the search. It commonly consists of tours recently encountered when moving from one tour to another, which remain consequently forbidden for a certain amount of time (iterations).

In order to reduce storage requirements, usually a set of rules, such as a collection of forbidden moves, are stored in the list instead of complete banned tours. The list is called Tabu list. Thus, finding an appropriate way to describe the forbidden tours in a compact structure is also part of the design of a TS algorithm. In this work, the attribute we store in the list is the pair of customers swapped by the 2-opt heuristic (Section 4.3).

During the searching procedure, each time we select the best non-forbidden tour in the neighborhood $\mathcal{N}(\tau)$ as the incumbent solution τ_{best} (no matter it is better or worse than current best solution). The two customers j and k between which the route is reversed by the 2-opt operation transforming τ into τ_{best} are stored in the Tabu list. Any 2-opt operation involving customers in the Tabu list as pivots j and k is forbidden.

The length of the Tabu list l (also known as Tabu tenure) decides the number of Tabu Search iterations for which a pair of customers remain forbidden. l pairs of customers are forbidden to move at the same time when the Tabu list is full, thus the maximum value of l should be less than $\frac{1}{2}n$. In Tabu Search implementations the Tabu tenure can either be a fixed number or a flexible value that changes deterministically with problem parameters (Basu, 2012). In this work, we test both fixed and flexible values, where l is given by a coefficient $f \in [0, 0.5]$ multiplied by the number of customers n . Detailed experimental results about the tuning of parameter l can be found in Section 5.

As mentioned in Section 4.3, in the neighbourhood $\mathcal{N}(\tau)$ of a complete tour τ , several tours may actually correspond to the same POP solution. Therefore, the Tabu list applies only for the q customers visited before the deadline, namely the neighbourhood of $\mathcal{N}(\sigma(\tau))$.

4.5. The complete Tabu Search algorithm

A pseudo code for our approach is outlined in Algorithm 1.

The input of the algorithm is a POP instance. First a complete tour τ_0 is generated as initial tour in a way described in Section 4.2.

By using the Monte Carlo evaluator described in Section 4.1, a corresponding feasible solution $\sigma(\tau_0)$ is obtained by deciding to visit q customers. The value of q is indicated as $|\sigma(\tau_i)|$ in the algorithm. We then apply a 2-opt local search (Section 4.3) to τ_0 , by considering a group of neighboring tours $\mathcal{N}(\tau_0)$. A memory structure (Section 4.4) is used to prevent the local search from re-visiting the same tours that has been visited recently. For each neighboring tour $\tau_i \in \mathcal{N}(\tau_0)$ that has not been visited recently, its objective function value $u(\tau_i)$, the corresponding feasible solution $\sigma(\tau_i)$ and the objective function value $u(\sigma(\tau_i))$ are calculated by the Monte Carlo evaluator, and the current best tour τ_{local} is updated consequently.

In the algorithm, the local search proceeds in two steps. First, we swap between the customers that can be visited before the deadline and compare the objective function value $u(\sigma(\tau_i))$. The pair of customers $(\tau_0[j], \tau_0[k])$ in the position j and k that have been swapped to obtain the current best tour τ_{local} are saved consequently. After finish searching among the customers that can be visited before the deadline, a local search procedure without memory structure will run for the rest of the customers who are visited after the deadline incurred. The value of $u(\tau_i)$ is used here to discriminate between tours that differ only for the customers appearing after the deadline. In this way, when two different complete tours are associated to the same feasible solution, priority is given to the tour with a better tail after the customers included in the feasible solution. This strategy is very important in the economy of the meta-heuristic strategy we developed, since promising chunks of tours are prepared in complete tours after the end of the respective feasible solution, and these parts are used by the 2-opt component to improve the feasible solutions in subsequent iterations.

After comparing all applicable tours in $\mathcal{N}(\tau_0)$, a new group of neighboring tours $\mathcal{N}(\tau_{local})$ will be generated from the current best tour τ_{local} , and attributes of τ_{local} are stored in the memory structure. When no improving tour can be found after a certain number of iterations (set to 50 in this work), a new random tour is generated and the process is restarted until a given time limit is reached.

Some theoretical results about the number of tours left in the 2-opt neighbourhood when a Tabu list is implemented, are presented.

Given a complete tour τ with $q = |\sigma(\tau)|$ customers in the associated feasible solution $\sigma(\tau)$, and a Tabu list of length l containing $2 \cdot l$ distinct customers, the total number of meaningful tours left in the 2-opt neighbourhood $\mathcal{N}^l(\sigma(\tau))$ is given by the number of combinations of picking 2 customers out of $q - 2l$ customers:

$$L = |\mathcal{N}^l(\sigma(\tau))| = \frac{(q-2l)(q-2l-1)}{2} \quad (6)$$

L represents the number of meaningful tours left in the neighbourhood when a Tabu list of length l is full. In general we expect $L \gg 0$ in order to ensure that the search space is not (almost) empty, and the search can proceed fruitfully.

The actual value $q = |\sigma(\tau)|$ for the certain complete tour τ is given by the Monte Carlo evaluator. Given the value q and a Tabu list length l , L can be easily calculated, and the number of meaningful tours in the 2-opt neighbourhood can be easily checked. A closed formula for the total number of non-forbidden tours of the 2-opt neighbourhood of a specified complete tour τ forbidden by the Tabu mechanism is therefore given by:

$$|\mathcal{N}(\sigma(\tau))| - |\mathcal{N}^l(\sigma(\tau))| = (2q - 2l - 1)l \quad (7)$$

Algorithm 1: Tabu Search algorithm

```

Tabulist = [];
iterations = 0;
 $\tau_0 = \text{k\_Nearest\_Random\_Insertion}()$ ;
 $\tau_{best} = \tau_{current} = \tau_0$ ;
while runtime  $\leq$  max_runtime do
     $\tau_{local} = -\infty$ ;
    for  $j = 1, \dots, |\sigma(\tau_{current})| + 1$  do
        for  $k = j + 1, \dots, |\sigma(\tau_{current})| + 1$  do
            if  $(\tau_{current}[j] \notin \text{Tabulist}) \ \& \ (\tau_{current}[k] \notin \text{Tabulist})$  then
                 $\tau_{j,k} = \text{2-opt}(\tau_{current})$ ;
                if  $u(\sigma(\tau_{j,k})) > u(\sigma(\tau_{local}))$  then
                     $\tau_{local} = \tau_{j,k}$ ;
                     $(c_1, c_2) = (\tau_{current}[j], \tau_{current}[k])$ ;
                end
            end
        end
    end
     $\tau_{current} = \tau_{local}$ ;
    for  $j = |\sigma(\tau_{current})| + 1, \dots, n + 1$  do
        for  $k = j + 1, \dots, n + 1$  do
             $\tau_{j,k} = \text{2-opt}(\tau_{current})$ ;
            if  $u(\sigma(\tau_{j,k})) > u(\sigma(\tau_{local}))$  then
                 $\tau_{local} = \tau_{j,k}$ ;
            end
        end
    end
    if  $u(\sigma(\tau_{local})) > u(\sigma(\tau_{best}))$  then
         $\tau_{best} = \tau_{local}$ ;
        iterations = 0;
    end
    iterations = iterations + 1;
    if iterations  $\geq$  max_non_improving_iterations then
         $\tau_{current} = \text{k\_Nearest\_Random\_Insertion}()$ ;
        iterations = 0;
    end
    Insert  $(c_1, c_2)$  in the Tabulist;
    if Tabulist is full then
        remove oldest pair;
    end
end
return  $\sigma(\tau_{best})$ ;

```

5. Experimental results

In this section we test the performance of the TS algorithm. We first tune the length of the Tabu list, and then compare the algorithm with other heuristic methods for the POP.

5.1. Benchmarks and settings

The POP benchmarks used for the main experiments in this work are the 264 instances generated by Angelelli et al. (2017).¹

They are based on 22 TSP benchmark instances originally from the TSPLIB95 library.²

The characteristics of the instances are set as follows:

- The customers information and distances are taken directly from the corresponding original TSP instances. The first customer of the original instance is now considered as the depot $V_0 = 0$. The destination node $V_{n+1} = n + 1$ coincides with V_0 .

¹ The POP instances are available at <http://or-brescia.unibs.it/instances/>

² The TSPLIB library can be found at <http://iwr.uni-heidelberg.de/groups/comopt/software/TSPLIB95/>

- The global Deadline D takes value of $\omega \cdot T_{max}$, with T_{max} being the known optimal value of the original TSP instance and $\omega = \{\frac{1}{4}, \frac{1}{2}, \frac{3}{4}\}$ representing short, medium and long deadlines.
- The prizes collected while visiting the customers are either $p_i = 1$ for all customers, or p_i is a pseudo-randomly generated integer in $\{1, 2, \dots, 100\}$ for each customer i .
- The probabilities of the customers requiring visits are either $\pi_i = 0.5$ for all customers, or π_i is a random number in the interval $[0.25, 0.75]$ for each customer i .

The coefficient C of the objective function (2), used to balance between the travel time and the prize components, is taken as $C = 0.001$, in order to maintain full compatibility and comparability with the results of Angelelli et al. (2017), where the same setting is considered. The setting of the parameter l , regulating the length of the Tabu list (see Section 4.4) is more delicate, and it is discussed in details in Section 5.2.

The computation time limit for the TS method is finally set at 180 s unless differently stated. Since the method has been design to be a fast method, this threshold is enough in order not to limit its performance on the main benchmark set.

For each final solution σ retrieved by the algorithm, its objective function value is evaluated by an exact evaluation, as described in Angelelli et al. (2017):

$$u(\sigma) = \sum_{k=1}^q \pi_k p_k - C \sum_{h=0}^q \left[\pi_h \sum_{k=h+1}^{q+1} \left(t_{hk} \pi_k \prod_{j=h+1}^{k-1} (1 - \pi_j) \right) \right] \quad (8)$$

where it is assumed that $\prod_{j=h+1}^{k-1} (1 - \pi_j) = 1$ whenever $h + 1 > k - 1$.

The testing environment is computer equipped with an Intel Core i7-9700K processor running at 3.6 GHz and with 16 GB of RAM. A single thread was used for all the experiments. The algorithm we propose was implemented in C++.

5.2. Tuning for the length of the Tabu list

As alternatives for the length of the Tabu list l , in order to find a tuning that performs well in general case, four flexible values $l = f \cdot n$ are tested, with $f \in \{0.02, 0.05, 0.1, 0.15\}$ being a coefficient and n being the number of customers of the instance under investigation (note that the number of customers ranges between 15 and 97 for the 264 instances from Angelelli et al. (2017)).

The objective function value of the best solution retrieved with a certain Tabu list length l is denoted as X_l . Since an exact solution is not available for all the instances considered, the best solution reported in Angelelli et al. (2017) for each instance is considered as a reference value X_{ref} . We calculate, for each setting l , the error

in percentage by computing the relative difference between X_l and X_{ref} :

$$error(\%) = 100 \left| \frac{X_l - X_{ref}}{X_{ref}} \right| \quad (9)$$

For an indepth and more precise study, the 264 instances are divided into different groups based on their characteristics. Such a classification has already proven significant in previous studies (Angelelli et al., 2017; Chou et al., 2018, 2019a,b). According to the dimension, there are 84 small instances with $n < 30$, 84 medium instances with $30 \leq n < 50$, and 96 large instances with $n \geq 50$. In terms of the deadline, there are also three groups. For Prize type and Probability type, there are two groups with 132 instances for each. In the experiment, for each Tabu list length l , we report the average and best result over 5 runs.

Table 1 shows the average error and computation time over 5 runs for different values of l . The last row of the table shows the average results for all the 264 instances. In Table 1 we observe that the choice of the Tabu list length l is different for each dimension group. For small instances with $n < 30$, the best result is obtained with $l = 0.15n$, for medium instances with $30 \leq n < 50$ and large instances with $n \geq 50$, the best result is obtained with $l = 0.05n$. It is worth noting that, when $l = 0.02n$, the Tabu list length is 0 for some of the small instances ($n < 30$). In this case the TS mechanism simply does not get activated. Apart from this, smaller values of l appear more indicated for larger n . This shows that the choice of l might be independent of n . In particular, fixed values like 2 or 3 seem to be a good choice according to the emerging trend.

As mentioned in Section 4, the memory structure works only on the q customers visited before the deadline. The value of q is mainly influenced by the instance size and deadline type. From the table we can also observe that the algorithm performs differently on relevant subgroups. Among the given POP benchmarks, the actual value of q varies from 4 to 21 for small instances, from 7 to 40 for medium instances, and from 1 to 85 for large instances. Considering Eqs. (5) and (7), when appropriate values of Tabu list lengths are taken (e.g. $l = 2$), 57.6% of meaningful complete tours on average are forbidden by the Tabu mechanism for small instances, 32.5% for medium instances, and 20.2% for large instances. Theoretical and empirical results are therefore consistent.

In general, a smaller value of Tabu list length requires longer computation time, and the average computing time is less than 10 s for all settings. This shows that the TS algorithm we propose is able to give a relatively good result for a POP instance within a very short time.

Table 1
Average results over 5 runs for the TS algorithm with different tabu lengths.

instances\l	0.02n		0.05n		0.1n		0.15n	
	error (%)	time (s)	error (%)	time (s)	error (%)	time (s)	error (%)	time (s)
$n < 30$	0.059	0.16	0.055	0.15	0.040	0.21	0.029	0.24
$30 \leq n < 50$	1.683	6.25	1.618	5.37	1.707	4.46	2.201	4.31
$n \geq 50$	6.172	10.82	6.012	11.10	6.871	6.73	8.420	4.26
$1/4T_{max}$	1.313	3.59	1.225	2.89	1.803	1.86	2.829	1.44
$2/4T_{max}$	3.626	6.31	3.507	6.31	3.681	4.64	4.522	3.19
$3/4T_{max}$	3.458	8.04	3.425	8.17	3.679	5.30	3.964	4.36
$\pi_i = 1$	2.724	5.78	2.721	5.62	3.053	3.91	3.834	2.95
$\pi_i = random$	2.874	6.18	2.716	5.96	3.056	3.96	3.710	3.05
$p_i = 0.5$	2.720	5.97	2.583	5.80	3.0345	4.24	3.662	2.91
$p_i = random$	2.878	5.99	2.854	5.78	3.074	3.63	3.881	3.08
ALL	2.800	5.98	2.719	5.79	3.054	3.93	3.772	3.00

In Table 2 we present the best results obtained by the TS algorithm. For each characteristic group, the error with the corresponding number of best known solutions found bs (comparing to the best solutions disclosed in Angelelli et al., 2017) and the computation time are reported. Again, the last row contains the average results over all the instances. The best choice of Tabu list length is again slightly different regarding different characteristics. The algorithm performs extremely well on small instances with $n < 30$, with $l = 0.15n$ being the best with an error of 0.024% and 79 out of 84 instances solved to optimal. For medium size instances with dimension $30 \leq n < 50$, $l = 0.1n$ performs the best, the problem can be solved with an error of 0.964%. For large instances with dimension $n \geq 50$, $l = 0.05n$ performs the best with an error of 4.255%. For all the rest of characteristics $l = 0.05n$ performs the best, with 30–40% of the meaningful complete tours forbidden. Therefore, we choose $l = 0.05n$ for the comparison with other heuristic methods from the literature provided in Section 5.3.

5.3. Comparison with other methods

In this section we compare the general performance of the TS algorithm we propose with other heuristic methods for the POP. For each method we present the error in percentage (again with reference to the best results published in Angelelli et al. (2017)), the number of best solutions found and the computing time for each of the characteristic groups, as well as the aggregated results for all the 264 instances.

Three matheuristics based on a branch-and-cut framework are proposed by Angelelli et al. (2017): Smart-Chain (**sChain**), Smart-

Path (**sPath**), and Smart-Two-ways-path (**s2Path**). The Chain, Path and Two-ways-path are alternative branching strategies proposed for the POP focusing on chains starting in 0 or ending in $n + 1$. The corresponding matheuristics are branch-and-cut algorithms with heuristic branching and variable fixing to reduce the solution space. We refer the interested reader to Angelelli et al. (2017) for complete details of the methods. The experiments presented in Angelelli et al. (2017) are carried out on a Intel Xeon E5-1650 processor, 3.50 GHz and 16 GB of RAM with a max computation time of 7200 s. In order to have here a fair comparison, we normalized the computation times from Angelelli et al. (2017) to our machine, which according to <http://cpubenchmark.net> is faster by a factor of about 38%. The results are reported in Table 3, where the results of TS are obtained with $l = 0.05n$ (see Section 5.2 and a time limit again of 180 s.

From Table 3 we can observe that the TS algorithm performs very well on instances with small dimension and short deadline. For instances with dimension less than 30, the TS algorithm retrieves more best known solutions with much shorter times when compared to the other methods. For medium size instances, the TS algorithm is competitive with the other methods on accuracy and computing time. sChain method performs the best on error, but also requires longer computing times. For large instances, the matheuristics from Angelelli et al. (2017) are much more accurate, but with longer computational times. This shows that the 2-opt local search used in TS algorithm might not be sufficient, and stronger local search methods might lead to more accurate results. The TS algorithm remains anyway always much faster.

In general, we can conclude that the TS algorithm is extremely fast, and with small errors. In practical terms, TS could be a pre-

Table 2
best result obtained by TS algorithm for different tabu lengths.

instances\l	0.02n			0.05n			0.1n			0.15n		
	error	bs	time	error	bs	time	error	bs	time	error	bs	time
$n < 30$	0.049	74	0.10	0.024	77	0.09	0.024	73	0.08	0.024	79	0.12
$30 \leq n < 50$	1.207	41	5.99	1.032	39	6.29	0.964	41	4.99	1.491	32	4.03
$n \geq 50$	4.337	16	11.00	4.255	20	11.66	5.016	12	8.73	5.894	12	5.84
$1/4T_{max}$	0.703	60	3.57	0.631	66	2.70	0.944	62	2.25	1.711	55	1.52
$2/4T_{max}$	2.626	42	6.33	2.498	42	7.27	2.676	38	5.65	3.392	39	3.86
$3/4T_{max}$	2.602	29	7.92	2.521	28	8.84	2.795	26	6.47	2.773	29	4.95
$\pi_i = 1$	1.696	75	6.78	1.784	77	6.37	1.935	73	4.82	2.443	69	3.87
$\pi_i = random$	2.258	56	5.10	1.983	59	6.16	2.341	53	4.75	2.807	54	3.02
$p_i = 0.5$	1.978	67	5.99	1.755	69	6.41	2.101	65	5.33	2.591	61	3.37
$p_i = random$	1.975	64	5.89	2.012	67	6.13	2.176	61	4.24	2.660	62	3.53
ALL	1.977	131	5.94	1.883	136	6.27	2.138	126	4.79	2.625	123	3.45

Table 3
Comparison between different heuristics for the POP.

instances\methods	sChain (Angelelli et al., 2017)			sPath (Angelelli et al., 2017)			s2Path (Angelelli et al., 2017)			TS		
	error	bs	time	error	bs	time	error	bs	time	error	bs	time
$n < 30$	2.52	63	21	3.93	56	120	2.72	60	168	0.02	77	0.09
$30 \leq n < 50$	0.38	49	27	0.75	37	6	1.05	34	5	1.03	39	6.29
$n \geq 50$	0.77	41	223	1.17	26	181	1.34	23	127	4.26	20	11.66
$1/4T_{max}$	1.02	56	40	1.25	50	11	1.57	48	14	0.63	66	2.70
$2/4T_{max}$	0.23	54	22	0.63	37	94	0.58	39	17	2.50	42	7.27
$3/4T_{max}$	0.35	43	232	0.61	32	209	0.65	30	288	2.52	28	8.84
$\pi_i = 1$	0.78	65	136	0.88	58	174	1.11	54	153	1.78	77	6.37
$\pi_i = random$	0.29	88	60	0.78	61	36	0.75	63	60	1.98	59	6.16
$p_i = 0.5$	0.37	82	126	0.72	58	127	0.87	61	123	1.75	69	6.41
$p_i = random$	0.70	71	71	0.94	61	82	1.00	56	89	2.01	67	6.13
ALL	0.53	153	98	0.83	119	104	0.93	117	107	1.88	136	6.27

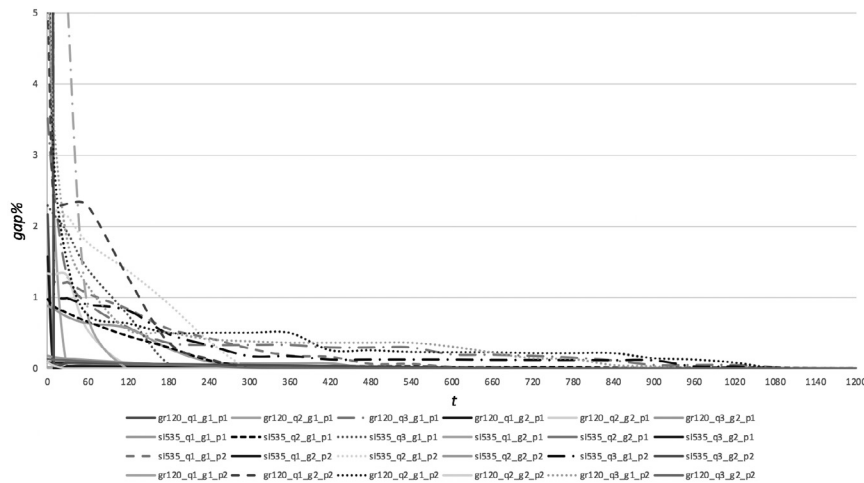


Fig. 2. converging speed of the example instances with different dimension.

cious approach for application where you need to take quick decisions (e.g. within an online decision support system, see Chou et al., 2019b).

5.4. Results on larger instances

The dimension of the 264 POP benchmarks for the previous experiments are up to 96 customers. In order to have a vision of how our approach performs on instances with more than 100 nodes, we generated more POP instances from the TSPLIB95 in the same way as described in Angelelli et al. (2017). The 24 new POP instances are based on 2 TSP instances of dimension 120 and 535, considering different characteristics (i.e. Deadline type q , Prize type g and Probability type p). The new instances are available upon request to the authors.

We solve the 24 POP instances with the TS algorithm presented in Algorithm 2. The computation time limit is set at 1200 s for these experiments. To show the converging speed, we calculate the relative difference (%) between the objective function value $u(\sigma_t)$ of the best feasible solution σ_t retrieved at a certain time t and the objective function value $u(\sigma_{1200})$ of the best solution found at time limit, as follows:

$$\text{gap}\%(t) = 100 \left| \frac{u(\sigma_t) - u(\sigma_{1200})}{u(\sigma_{1200})} \right| \quad (10)$$

we use $u(\sigma_{1200})$ as a reference because no other known solution is available.

The converging speed of the 24 new POP instances are shown in Fig. 2, with $\text{gap}\%(t)$ of the instance on y-axis and the time t on x-

axis. We can observe that the differences drop below 3% within 60s and below 1% within 180 s for all the example instances. This shows that the computation converges very fast in general even for large instances, indicating that scalability is promising for the method. We speculate that when larger instances without requirements for extremely high accuracy are considered, the TS can be a promising approach. The relevant best results found by the TS algorithm are reported in Table 4, for completeness and in case future researcher should use the new instances.

6. Conclusion

In this work we solved the Probabilistic Orienteering Problem by using a Tabu Search algorithm based on a Monte Carlo Sampling objective function evaluator. The integration of the two methods allows the design of a simple Tabu Search metaheuristic. Computational studies on the performance of the algorithm both in terms of precision and speed were carried out. With appropriate parameter tuning, the Tabu Search algorithm is overall extremely fast, and with errors comparable with those of considerably more complex heuristics from the literature.

CRedit authorship contribution statement

Xiaochen Chou: Conceptualization, Formal analysis, Data curation, Investigation, Methodology, Software, Writing - original draft. **Luca Maria Gambardella:** Conceptualization, Methodology, Writing - review & editing. **Roberto Montemanni:** Supervision, Validation, Writing - review & editing.

Acknowledgement

Xiaochen Chou was supported by the Swiss National Science Foundation through grant 200020-182360: “Machine learning and sampling-based metaheuristics for stochastic vehicle routing problems”.

References

- Angelelli, E., Archetti, C., Vindigni, M., 2014. The clustered orienteering problem. *European Journal of Operational Research* 238 (2), 404–414.
- Angelelli, E., Archetti, C., Filippi, C., Vindigni, M., 2017. The probabilistic orienteering problem. *Computers and Operations Research* 81, 269–281.
- Archetti, C., Hertz, A., Speranza, M., 2007. Metaheuristics for the team orienteering problem. *Journal of Heuristics* 13 (1), 49–76.

Table 4

best results found for the 24 new POP instances.

instance	result	instance	result
gr120_q1_g1_p1	57.68	si535_q1_g1_p1	85.94
gr120_q1_g1_p2	56.20	si535_q1_g1_p2	82.93
gr120_q1_g2_p1	2904.35	si535_q1_g2_p1	4628.16
gr120_q1_g2_p2	2815.31	si535_q1_g2_p2	4502.85
gr120_q2_g1_p1	56.91	si535_q2_g1_p1	84.01
gr120_q2_g1_p2	56.02	si535_q2_g1_p2	82.71
gr120_q2_g2_p1	2936.01	si535_q2_g2_p1	4614.98
gr120_q2_g2_p2	2824.68	si535_q2_g2_p2	4481.41
gr120_q3_g1_p1	57.29	si535_q3_g1_p1	83.90
gr120_q3_g1_p2	55.76	si535_q3_g1_p2	84.87
gr120_q3_g2_p1	2902.13	si535_q3_g2_p1	4628.27
gr120_q3_g2_p2	2768.52	si535_q3_g2_p2	4521.03

- Basu, S., 2012. Tabu search implementation on traveling salesman problem and its variations: A literature survey. *American Journal of Operations Research* 2, 163–173.
- Bérubé, J.F., Gendreau, M., Potvin, J.Y., 2009. An exact ϵ -constraint method for bi-objective combinatorial optimization problems: Application to the traveling salesman problem with profits. *European Journal of Operational Research* 194, 39–50.
- Blazinski, A., Misevicius, A., 2011. Combining 2-opt, 3-opt and 4-opt with k-swap-kick perturbations for the traveling salesman problem. In: *Proceedings of the 17th International Conference on Information and Software Technologies*, pp. 50–401.
- Bräysy, O., Gendreau, M., 2015. Vehicle routing problem with time windows, part i: Route construction and local search algorithms. *Transportation Science* 39 (1), 104–118.
- Campbell, A.M., Thomas, B.W., 2008. Probabilistic traveling salesman problem with deadlines. *Transportation Science* 42 (1), 1–21.
- Campbell, A.M., Gendreau, M., Thomas, B.W., 2011. The orienteering problem with stochastic travel and service times. *Annals of Operations Research* 186, 61–81.
- Chao, I., Golden, B., Wasil, E., 1996. Theory and methodology – the team orienteering problem. *European Journal of Operational Research* 88, 475–489.
- Chou X., Gambardella L.M., Montemanni, R., 2018. Monte Carlo sampling for the probabilistic orienteering problem. In: Daniele P., Scrimali L. (Eds.), *New Trends in Emerging Complex Real Life Problems*. AIRO Springer Series, vol. 1. Springer, Cham, pp. 169–177.
- Chou, X., Gambardella, L.M., Montemanni, R., 2019a. A metaheuristic algorithm for the probabilistic orienteering problem. In: *MLMI 2019: Proceedings of the 2019 2nd International Conference on Machine Learning and Machine Intelligence*. Association for Computing Machinery, pp. 30–34.
- Chou, X., Montemanni, R., Gambardella, L.M., 2019b. Monte Carlo sampling for the tourist trip design problem. *Millenium* 10 (2), 83–90.
- Chou, X., Mele, U.J., Gambardella, L.M., Montemanni, R., 2020. Re-initialising solutions in a random restart local search for the probabilistic orienteering problem. In: *2020 IEEE 7th International Conference on Industrial Engineering and Applications (ICIEA)*. IEEE, pp. 159–163.
- Dorigo, M., Gambardella, L.M., 1997. Ant colonies for the traveling salesman problem. *BioSystems* 43 (2), 73–81.
- Feillet, D., Dejax, P., Gengreau, M., 2005. Travelling salesman problems with profits. *Transportation Science* 39, 188–205.
- Fischetti, M., Salazar, J., Toth, P., 1998. Solving the orienteering problem through branch-and-cut. *INFORMS Journal on Computing* 10, 133–148.
- Gambardella, L.M., Montemanni, R., Weyland, D., 2012. Coupling ant colony systems with strong local searches. *Applied Artificial Intelligence* 220 (3), 831–843.
- Gengreau, M., Laporte, G., Semet, F., 1998. A branch-and-cut algorithm for the undirected selective travelling salesman problem. *Networks* 32, 263–273.
- Glover, F., 1986. Future paths for integer programming and links to artificial intelligence. *Computers and Operations Research* 13 (5), 533–549.
- Golden, B., Levy, L., Vohra, R., 1987. The orienteering problem. *Naval Research Logistics* 34, 307–318.
- Holland, J.H., 1992. *Adaptation in Natural and Artificial Systems: An Introductory Analysis with Applications to Biology, Control and Artificial Intelligence*. MIT Press Cambridge, MA, USA, ISBN 0262082136.
- Hoos, H.H., Stützle, T., 2005. Generalised local search machines. In: *Stochastic Local Search*. The Morgan Kaufmann Series in Artificial Intelligence. Elsevier, pp. 113–147.
- URL: <https://www.cpubenchmark.net/>; PassMark Software.
- Ilhan, T., Irvani, S.M.R., Daskin, M.S., 2008. The orienteering problem with stochastic profits. *IIE Transactions* 40 (4), 406–421.
- Keller, C.P., 1989. Algorithms to solve the orienteering problem: A comparison. *European Journal of Operational Research* 41 (2), 224–231.
- Kirkpatrick, S., Gelatt, C.D., Vecchi, M.P., 1983. Optimization by simulated annealing. *Science* 220, 671–680.
- Laporte, G., Martello, S., 1990. The selective travelling salesman problem. *Discrete Applied Mathematics* 26, 193–207.
- Mbiadou Saleu, R.G., Deroussi, L., Feillet, D., Grangeon, N., Quilliot, A., 2018. An iterative two-step heuristic for the parallel drone scheduling traveling salesman problem. *Networks* 72 (4), 459–474.
- Montemanni, R., Gambardella, L.M., 2009. Ant colony system for team orienteering problems with time windows. *Foundations of Computing and Decision Sciences* 34 (4), 287–306.
- Montemanni, R., Weyland, D., Gambardella, L.M., 2011. An enhanced ant colony system for the team orienteering problem with time windows. In: *2011 International Symposium on Computer Science and Society*, pp. 381–384.
- Montemanni R., D'ignazio F., Chou X., Gambardella L.M., 2018. Machine learning and Monte Carlo sampling for the probabilistic orienteering problem. In: *2018 Joint 10th International Conference on Soft Computing and Intelligent Systems (SCIS) and 19th International Symposium on Advanced Intelligent Systems (ISIS)*. IEEE, 2018, pp. 14–18.
- Papapanagiotou, V., Montemanni, R., Gambardella, L.M., 2015. Hybrid sampling-based evaluators for the orienteering problem with stochastic travel and service times. *Journal of Traffic and Logistics Engineering* 3 (2), 108–114.
- Ramesh, R., Yoon, Y., Karwan, M., 1992. An optimal algorithm for the orienteering tour problem. *ORSA Journal on Computing* 4, 155–165.
- Shaw P., 1997. A new local search algorithm providing high quality solutions to vehicle routing problems.
- Souffriau, W., Vansteenwegen, P., Vertommen, J., Berghe, G.V., Oudheusden, D.V., 2008. A personalised tourist trip design algorithm for mobile tourist guides. *Applied Artificial Intelligence* 22 (10), 964–985.
- Tang, H., Miller-Hooks, E., 2005a. A tabu search heuristic for the team orienteering problem. *Computer and Operations Research* 32, 1379–1407.
- Tang, H., Miller-Hooks, E., 2005b. Algorithms for a stochastic selective travelling salesperson problem. *Journal of the Operational Research Society* 56 (4), 439–452.
- Vansteenwegen, P., Oudheusden, D.V., 2007. The mobile tourist guide: An or opportunity. *OR Insights* 20 (3), 21–27.
- Vansteenwegen, P., Souffriau, W., Berghe, G.V., Oudheusden, D.V., 2009. Iterated local search for the team orienteering problem with time windows. *Computers and Operations Research* 36 (12), 3281–3290.
- Vansteenwegen, P., Souffriau, W., Oudheusden, D.V., 2011. The orienteering problem: A survey. *European Journal of Operational Research* 209 (1), 1–10.
- Wang, Q., Golden, B., Wasil, E., 2008. Using a genetic algorithm to solve the generalized orienteering problem. In: Golden, B., Raghavan, S., Wasil, E. (Eds.), *The Vehicle Routing Problem: Latest Advances and New Challenges*, pp. 263–274.
- Weyland, D., Montemanni, R., Gambardella, L.M., 2013. Heuristics for the probabilistic traveling salesman problem with deadlines based on quasi-parallel Monte Carlo sampling. *Computers and Operations Research* 40 (7), 1661–1670.